Ans-to.Q.no-01 (a).

explain with example what a greedy choice is.

Ans:

A greedy choice involves making locally optimal decisions at each stage with the hope of finding a global optimum. It doesn't consider the biggen picture but focuses on immediate gains.

For example:

In the context of activity selection,

let's consider a set of activities with their start & finish time.

Activity	start time	finish time.
A	1	3
B	2	5
C	3	9
D	6	8
E	8	10

The greedy approach would start with activity A because it has the earliest finish time. After selecting A, it would move on to the next activity with the earliest finish time that doesn't conflict, which is activity D. The activities would be the A, D & E.

In the context of Huffman coding,
..... used for loseless data compression.

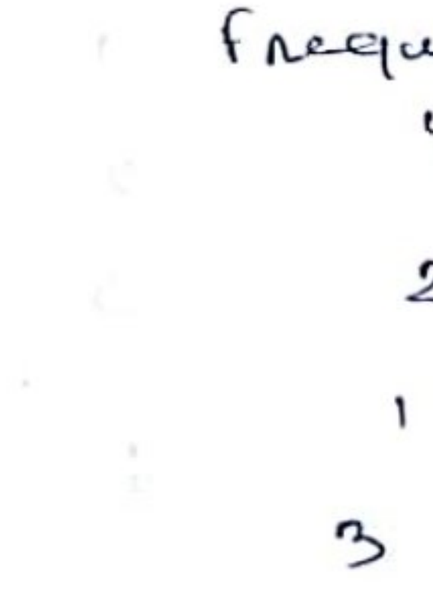
The simplified explanation of Huffman coding is stand with 4 base,

1. Frequency calculation.
2. Priority queue.
3. Tree construction
4. code assignment.

For example, consider the characters & their frequencies:

Character	frequency
A	4
B	2
C	1
D	3

The Huffman tree would be constructed as follows:



The code would be assigned as follows:

A: 0

B: 10

C: 110

D: 11

Huffman coding ensure that more frequent characters have shorter codes, optimizing the overall compression of the data.

Ans:- to. g. 20 - 01 (a → or)

Prove the activity selection problem has greedy choice property.

Ans:

The activity selection problem exhibits the greedy choice property, meaning that making a locally optimal choice at each step leads to a globally optimal solution. Let's consider a formal proof:

Greedy choice property:

Let's say we have a set of activities S sorted by their finish times. At each step, we select the activity with the earliest finish time among the remaining compatible activities.

Proof:

1. Base case:
considering the set of activities that finish first. The optimal solution must include the activity with the earliest finish time, which matches our greedy choice.

2. Inductive hypothesis:
Assume that at each step, we have made the greedy choice & selected the activity with the earliest finish time among the remaining compatible activities.

3. Inductive step:
Now, let's consider an arbitary step in the process. After selecting the first activity, we've removed any conflicting. Our inductive hypothesis implies that, the remaining set of activity is still optimal. Now, the earliest finish time among the remaining compatible activities is the globally optimal choice at this step.

4. conclusion:
By induction, the greedy choice at each step leads to globally optimal solution for the entire set of activities.

Therefore, the activity selection problem has greedy choice property.

Ans to Q.no – 01 (b)

The knapsack problem involves selecting a subset of items with given weights & values to maximize the total value without exceeding a certain weight capacity.

Here's How we can approach it:

1. Sort the note denominations in descending order: [20, 10, 5, 2, 1].
2. Initialite a variable to represent the remaining amount (x) to be paid.
3. Iterate through the note denominations:
   - for each dinomination, calculate how many times it can be used without exceeding the remaining amount.
   - update the remaining amount.
4. Output the result, showing the minimum number of each not denomination needed.

Here's a simple pseadocode for the greedy algorithm:

```
function payWithminimum_notes (x):
    note_denominations = [20, 10, 5, 2, 1]
    noks_used = [ ]
```

```
while x >= note:
    count = x / note
    notes_used.append((note, count))
    x -= count * note

return notes_used
```

Ans. to. Q. no — 01 (c)

As, ID : C213235

So, M = 3 + 5 = 8

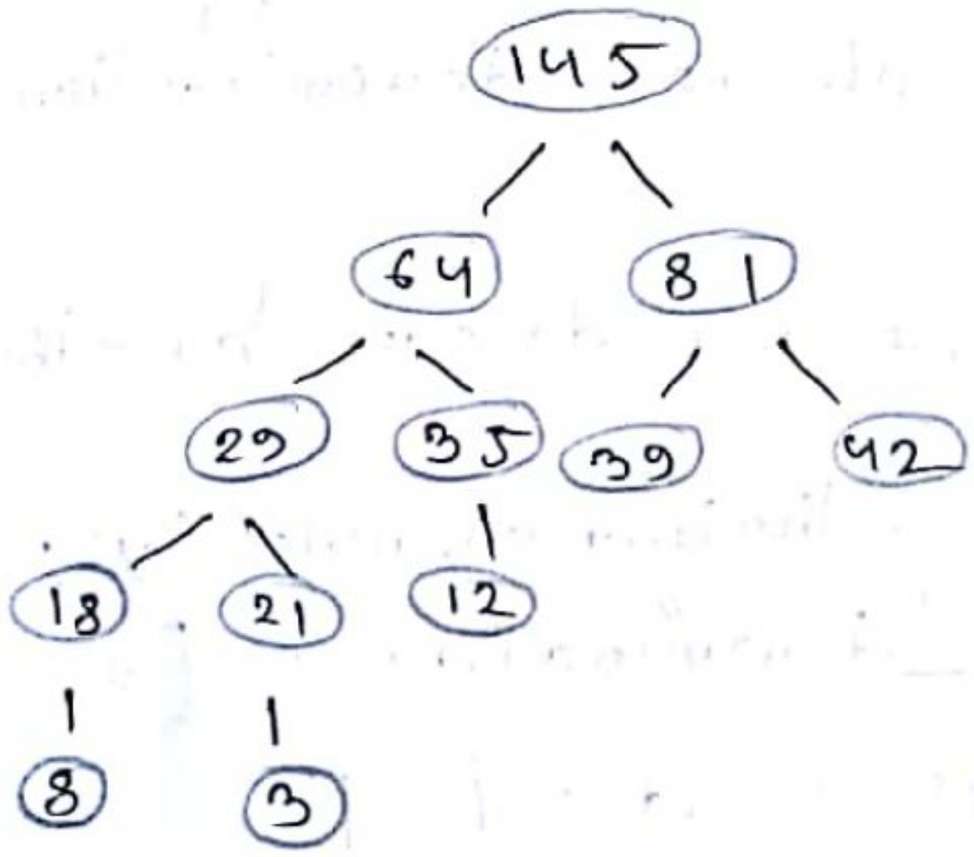
N = 35 .

∴ the frequencies are,

a : 8, b : 3, c : 18, d : 12, e : 35, f : 6,

g : 21, h : 42 .

So the huffman tree is :



Priority queue :

3, 6, 8, 12, 18, 21, 35, 42 .

1. Extract 3 & 6, create node with frequency 9, insert back into the queue,

8, 9, 12, 18, 21, 35, 42 .

2. Extract 8 & 9, create new node with frequency 17 insert back into the queue.

12, 17, 18, 21, 35, 42 .

3. Extract 12 & 17, create new node with frequency 29, insert back,

18, 21, 29, 35, 42 .

18, 21

4. Extract ~~29, 35~~, create new node with frequency 64, insert back,

29, 35, 39, 42 .

29 & 35

5. Extract , create new node with frequency 64, insert back,

~~64, 81~~ 39, 42, 64

6. Extract 39 & 42, create node with 81, insert back,

64, 81

7. Extract 64 & 81, create new node with length 145.

So the final codes will be,

8 : 000

3 : 0010

18 : 0011

12 : 010

35 : 011

6 : 1000

21 : 1001

42 : 101 .

Ans. to the Q. no - 02 (a)

Q. Analyze the running time of Prim's algorithm if the priority queue is represented by a binary heap.

⇒ Prim's algorithm used for finding the minimum spanning tree in a weighted, connected graph.

The running time of Prim's algorithm when the priority queue is represented by a binary heap is $O((E+V)\log V)$ where E is the number of edges and V is the number of vertices in the graph.

This time complexity accounts for the operations involved in the algorithm, such as inserting vertices, into the priority queue, updating the keys of vertices, and extracting the minimum key. The logarithmic factor arises from the operations on the binary heap, which include insertion, key updates and extraction of the minimum element.

Ans. to the Q.no - 02 (a) "ঘ"

Q. Analyze the running time of Dijkstra's Single Source Shortest algorithm if the priority queue is represented by a binary heap.

⇒ Dijkstra's single source shortest algorithm is used for finding the shortest paths from a single source vertex to all other vertices in a weighted graph.

The time complexity of Djkstra's algorithm uses a binary heap to represent the priority queue is $O((E+v) * \log(v))$, where v is the number of vertices and E is the number of edges in the graph.

The algorithm has a main loop that iterates through all vertices, and in each iteration, it performs operations such as extracting the minimum element from the priority queue (binary heap) and updating the distances. These operations take logarithmic time in the size of the priority queue, which is proportional to the number of vertices.

Ans. to the Q. no - 02 (b)

(i) Code segment for the in-degree of each vertex.

```
int IN_DEG[V+1];
void computeINDeg(int v)
{ int i,j;
  for(i=1; i<=v; i++)
    { for(j=1; j<=v; j++)
        { if(graph[j][i]==1)
            { IN_DEG[i]++;
            }
        }
    }
}
```

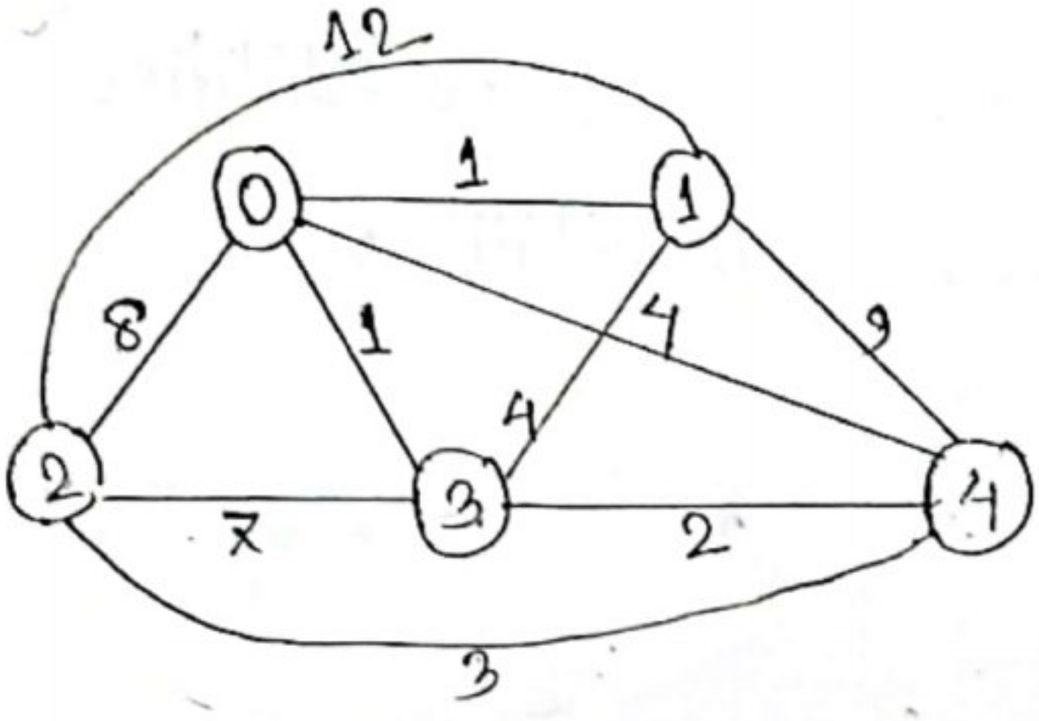
(ii) Code segment, whether there is any loop in the graph.

```
bool checkLOOP(int v) //returns true if there exists a loop
{ int i;
  for(i=1; i<=v; i++)
    { if(graph[i][i]==1)
        { return true;
        }
    }
 return false;
}
```

Ans. to the Q. no - 06

Here,
vertex set {0, 1, 2, 3, 4} and the graph is undirected.
leaf node : 0
Minimum possible weight of a spanning tree :

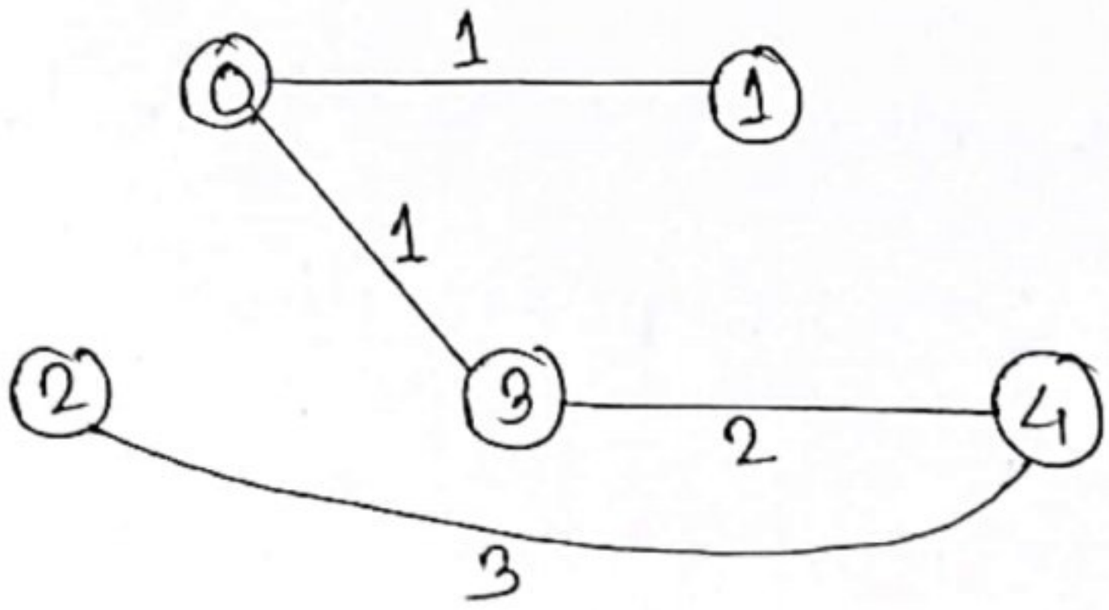
(I)



After removing loop & parallel edges.

(II) Sort the edges into ascending order.

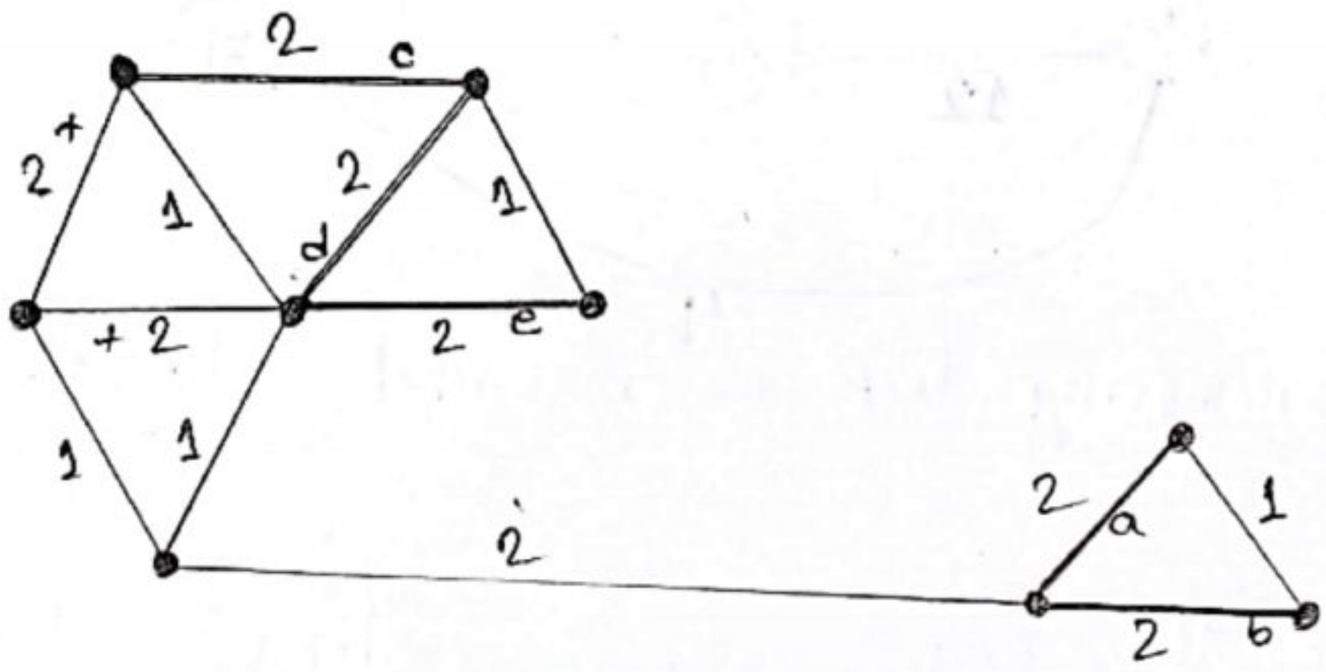
0,1	0,3	3,4	2,4	0,4	1,3	2,3	0,2	1,4	1,2
1	1	2	3	4	4	7	8	9	12
				x	x	x	x	x	x



(MST)

Ans. to the Q. no - 02 (c) "OR"

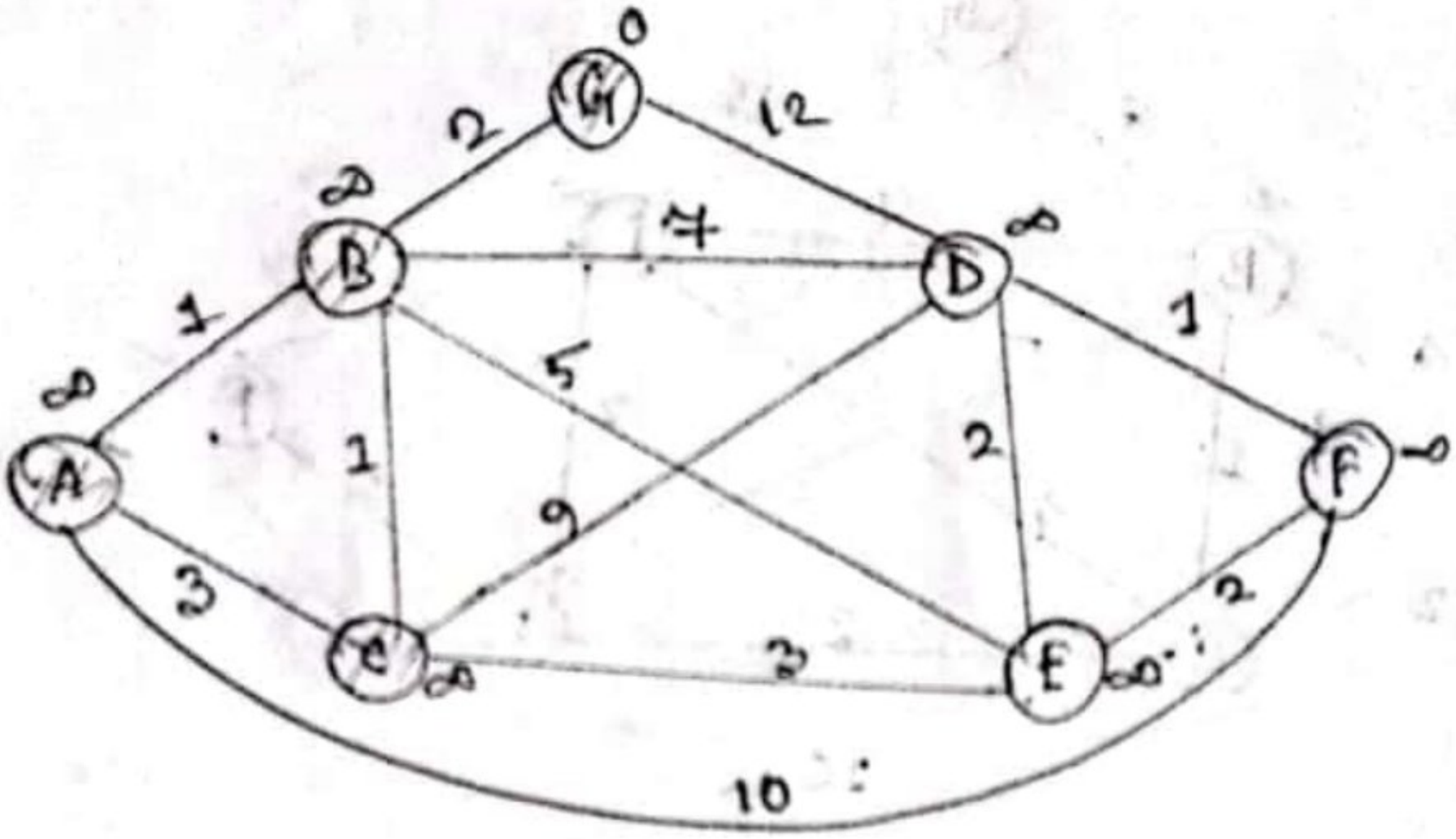
The number of distinct minimum spanning trees for the given weighted graph is $2 * 3 = 6$. There, in the right side we have 2 option & in the left side we have 3 option to pick the edges.



1. Kruskal's MST
2. Chose any one '2' from left side three '2's. (3 options)
3. chose any one '2' from right side two '2's. (2 options)
4. Remove other '2's not to create cycle
5. Total $3 * 2 = 6$ possibilities.

Ans: 6 distinct MST

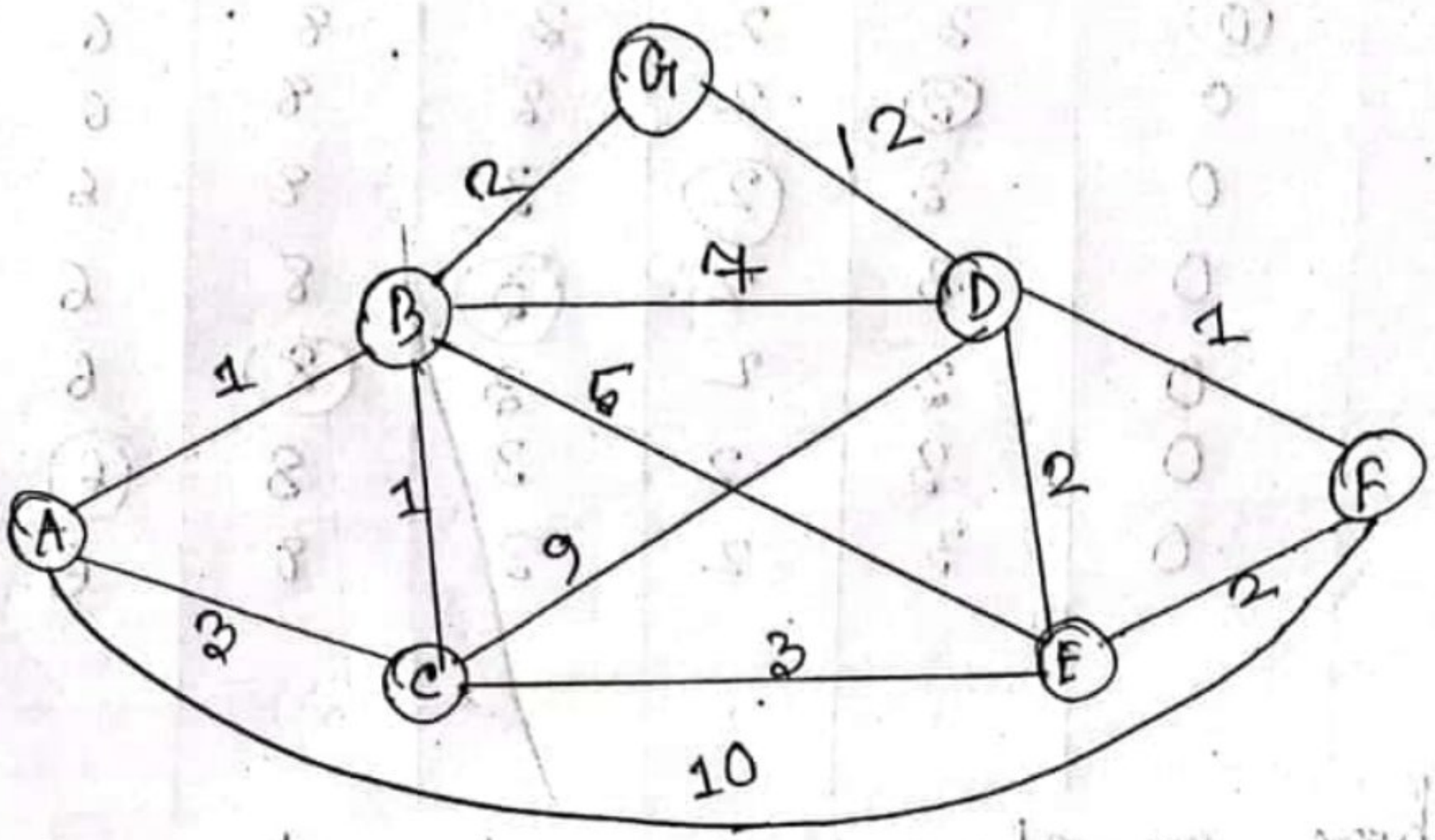
3(a)



Visited Vertex	G	A	B	C	D	E	F
	(0)	∞	∞	∞	∞	∞	∞
B		∞	(2)	∞	12	∞	∞
A		(3)		3	9	4	∞
C				(3)	9	4	13
E					9	(6)	13
D					(8)		8
F							(8)

∴ The lowest - cost path from G to F : F, E, C, B
and the result is G → B → C → E → F

3a(on)



visited vertices	G	A	B	C	D	E	F
G	(0)	∞	∞	∞	∞	∞	∞
B	0	2	(2)	∞	12	7	∞
C	0	3	2	(3)	9	7	∞
D	0	3	2	3	(9)	6	∞
E	0	3	2	3	9	(6)	10
F	0	3	2	3	8	6	(8)
A	0	(3)	2	3	8	6	8
	0	3	2	3	8	6	8

Second Iteration: [A.25]

V.V	G	A	B	C	D	E	F
G	(0)	3	2	3	8	6	8
A	0	(3)	2	3	8	6	8
B	0	3	(2)	3	8	6	8
C	0	3	2	(3)	8	6	8
D	0	3	2	3	(8)	6	8
E	0	3	2	3	8	(6)	8
F	0	3	2	3	8	6	(8)

There have no change consecutive two eqn on that. the other Third, fourth, Fifth and sixth iteration will also same output. After all iteration the shortest path is, the single source (G)

G → 0

A → 3

B → 2

C → 3

D → 8

E → 6

F → 8

3(b)

$D(4) =$

0	3	2	7	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

Case Know,

$$D^k[i,j] = \min\left\{ D^{k-1}[i,j],\ D^{k-1}[i,k] + D^{k-1}[k,j] \right\}$$

$$D^4[1,2] = \min\left\{ D^3[1,2],\ D^3[1,4] + D^3[4,2] \right\}$$

1-2
1-4, 4-2

$$= \min\{3,\ 7 + (-1)\}$$
$$= \min\{3, 6\}$$
$$= 3$$

$$D^4[1,3] = \min\left\{ D^3[1,3],\ D^3[1,4] + D^3[4,3] \right\}$$
$$= \min\{8,\ 7 + (-5)\}$$
$$= \min\{8, 2\}$$
$$= 2$$

$$D^4[1,5] = \min\{D^3[1,5],\ D^3[1,4] + D^3[4,5]\}$$
$$= \min\{-4,\ 7+(-2)\}$$
$$= \min\{-4,5\}$$
$$= -4$$

$$D^4[2,1] = \min\{D^3[2,1],\ D^3[2,4] + D^3[4,1]\}$$
$$= \min\{-4,\ 7+(-2)\}$$
$$= \min\{-4,5\}$$
$$= -4$$

$$D^4[2,3] = \min\{D^3[2,3],\ D^3[2,4] + D^3[4,3]\}$$
$$= \min\{\infty,\ 1+(-5)\}$$
$$= \min\{\infty,\ -4\}$$
$$= -4$$

$$D^4[2,5] = \min\{D^3[3,1],\ D^3[3,4] + D^3[4,1]\}$$
$$= \min\{\infty,\ 5+2\}$$
$$= \min\{\infty;\ 7\}$$
$$= 7$$

$$D^{\prime}[5,8] = \min\left\{D^{\beta}[5,8], D^{\beta}[5,4]+D^{\beta}[4,3]\right\}$$

$$= \min\left\{\infty, 6+(-5)\right\}$$

$$= \min\left\{\infty, 1\right\}$$

$$= 1$$

$$D^{\prime}[5,2] = \min\left\{D^{\beta}[5,2], D^{\beta}[5,4]+D^{\beta}[4,2]\right\}$$

$$= \min\left\{\infty, 6+(-1)\right\}$$

$$= \min\left\{\infty, 5\right\}$$

$$= 5$$

$$D^{\prime}[6,1] = \min\left\{D^{\beta}[5,1], D^{\beta}[5,4]+D^{\beta}[4,1]\right\}$$

$$= \min\left\{\infty, 6+2\right\}$$

$$= \min\left\{\infty, 8\right\}$$

$$= 8$$

$$D^{\prime}[8,5] = \min\left\{D^{\beta}[8,5], D^{\beta}[8,4]+D^{\beta}[4,5]\right\}$$

$$= \min\left\{11, 5+(-2)\right\}$$

$$= \min\left\{11, 3\right\}$$

4(a)

Given points,

A(5, 3)

B(17, 9)

C(2, 6)

D(5, 12)

Let, $B' = B - A$

$= (17-5, 9-3)$

$= (12, 6)$

and $C' = C - A$

$= (2-5, 6-3)$

$= (-3, 3)$

Now, $B' \times C' = \det\begin{pmatrix} 12 & -3 \\ 6 & 3 \end{pmatrix}$

$= 36 + 18 = 54$

Since $B' \times C'$ is positive, hence AB is clockwise from AC.

Therefore, the ant took left turn at point B.

Now, let,

$C'' = C - B$

$= (2-17, 6-9)$

$= (-15, -3)$

$D' = D - B$

$= (5-17, 12-9)$

$= (-12, 3)$

$C'' \times D' = \det\begin{pmatrix} -15 & -12 \\ -3 & 3 \end{pmatrix}$

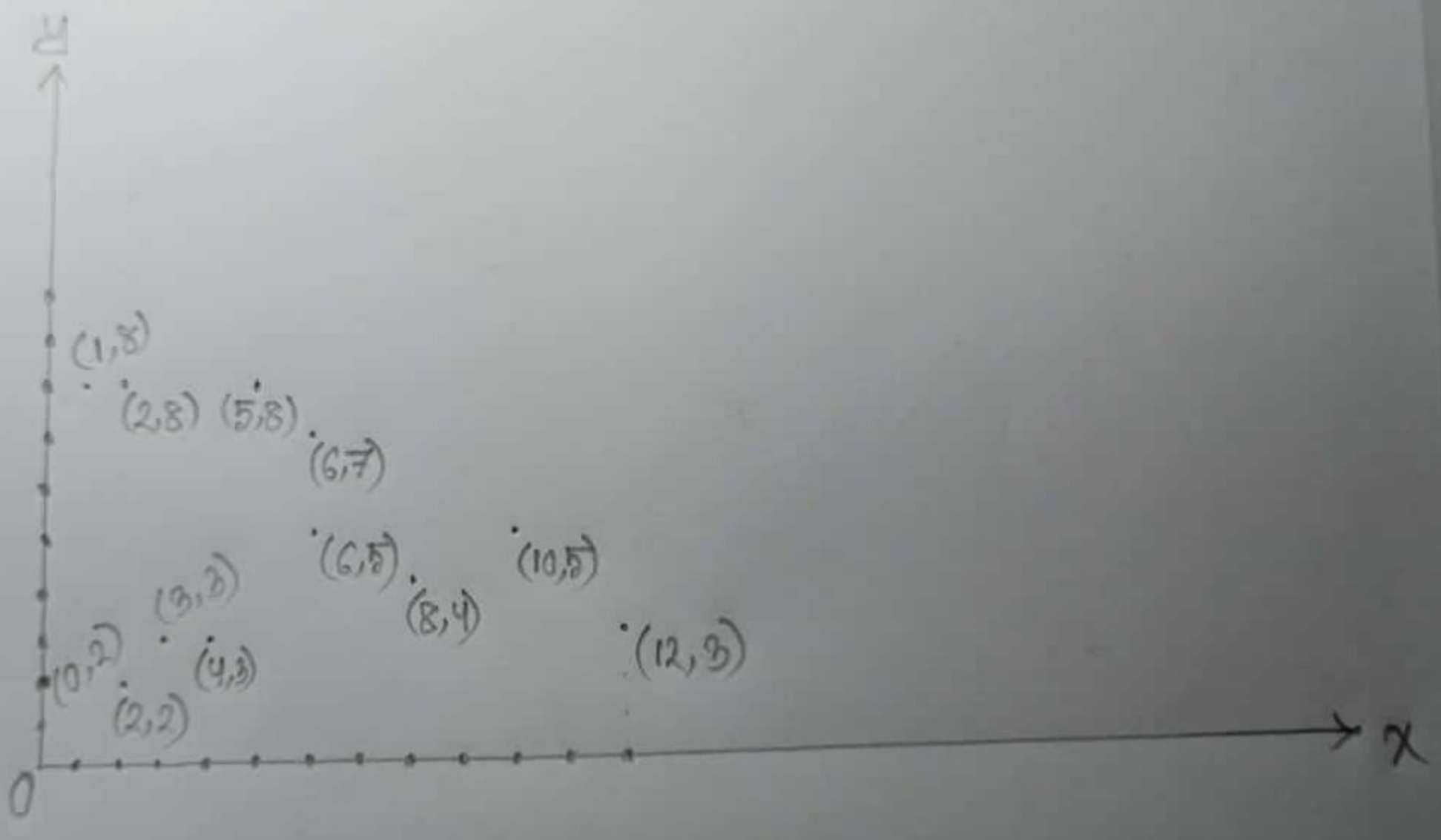
$= -45 - 36$

$= -81$

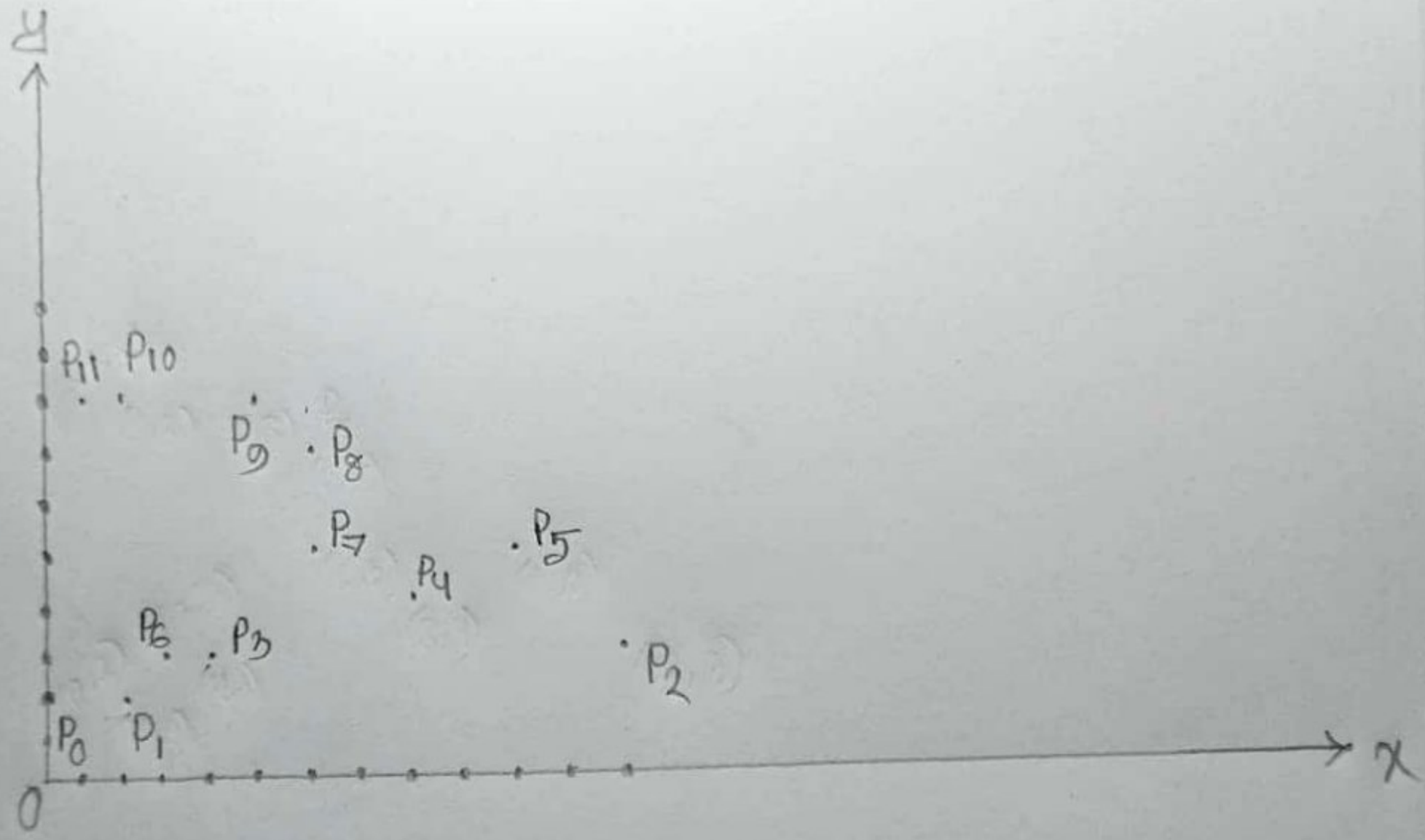
Since $C'' \times D'$ is negative, hence BC is counterclockwise from BD.

Therefore, the ant took right turn at point C.

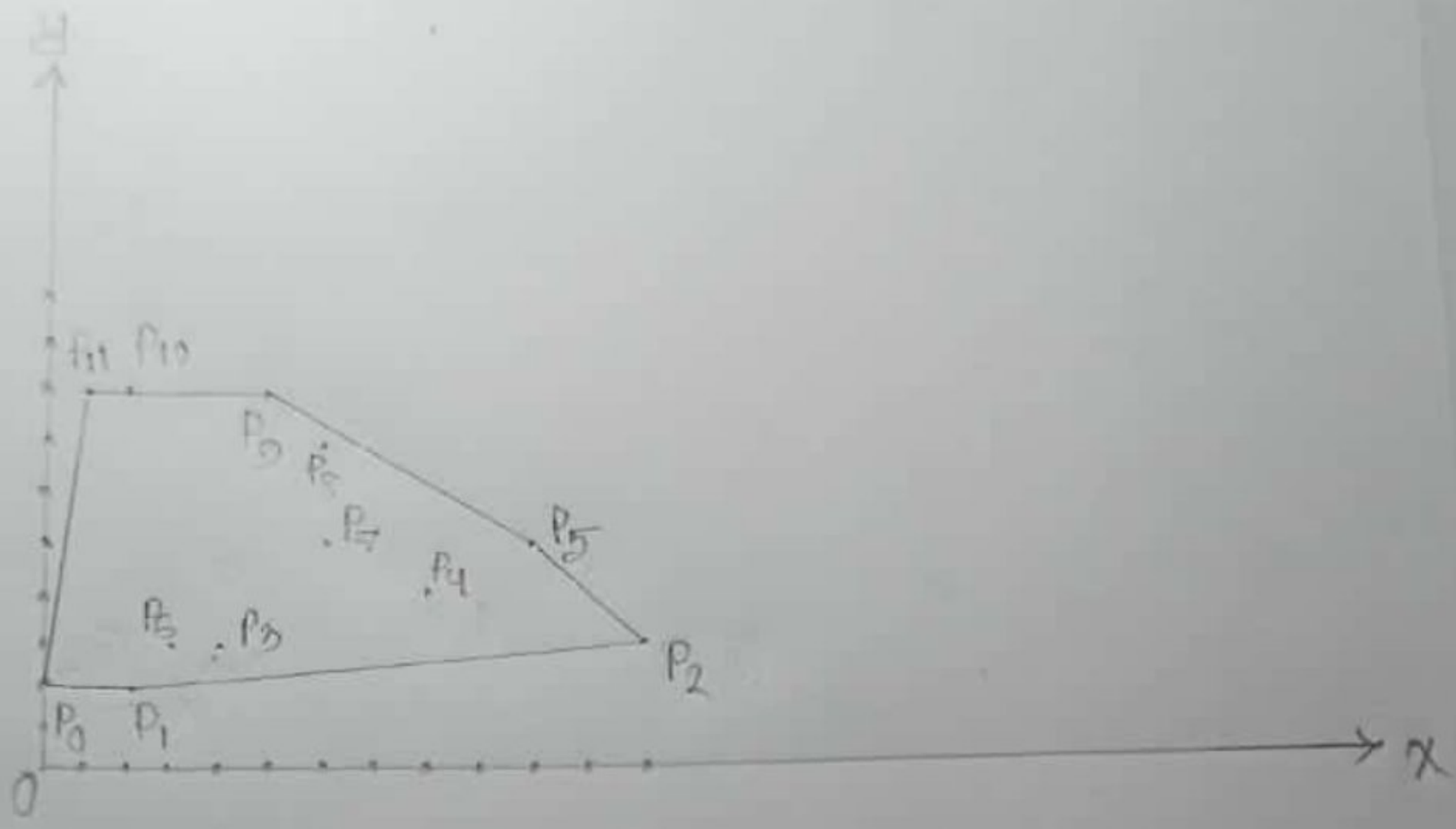
4(b)(i)



4(b)(ii)



4(b)(iii)



P_{11}
P_{10}
P_9
~~P_8~~
~~P_7~~
~~P_6~~
P_5
~~P_4~~
~~P_3~~
P_2
P_1
P_0

Stack

5(a)

NP- Complete problems are crucial in Complexity theory because solving any of them efficiently would means solving all problems in NP efficiendly, impacting various problems.

The NP complete class is a group of problems that are belived to be among the hardest problems to solve efficiently. It would mean that we have an efficient solution for all problem in the NP class. This is because NP complete problems are considered to be "equally hard" in the sense that if we can solve one, we can solve them all.

5(b)

The basic principle of reducing the solution/ state/search space in the branch and bound technique is to break down the optimization problem into smaller sub-proble and use bounding to eliminate certain sub-problems from further consideration.

The branch and bound technique involves the following steps:

Branching: The initial problem is divided into smaller sub-problems by making decisions at each step.

Bounding: A bounding function is used to estimate the potential of each sub-problem.

Backtracking: The algorithm explores the sub-problem in a depth-first. manner, keeping track of the current best solution.

Optimality: The algorithm continues until all sub-problem have been explored.

To illustrate this principle, let's consider an example of the Travelling salesman Problem (TSP) using the branch and bound technique. In TSP, the goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

Suppose, we have 4 cities : A, B, C, D.
The distance between these cities are :

Distance between A and B : 10
" " A and C : 5
" " A and D : 8
" " B and C : 3
" " B and D : 7
" " C and D : 1

The branch and bound technique can be used to find the optimal solution for this problem. The algorithm starts with an initial state where no cities have been visited. For example, it can branch out from the initial state to two sub-problems - one where city B is visited next
- another where city C is visited next.

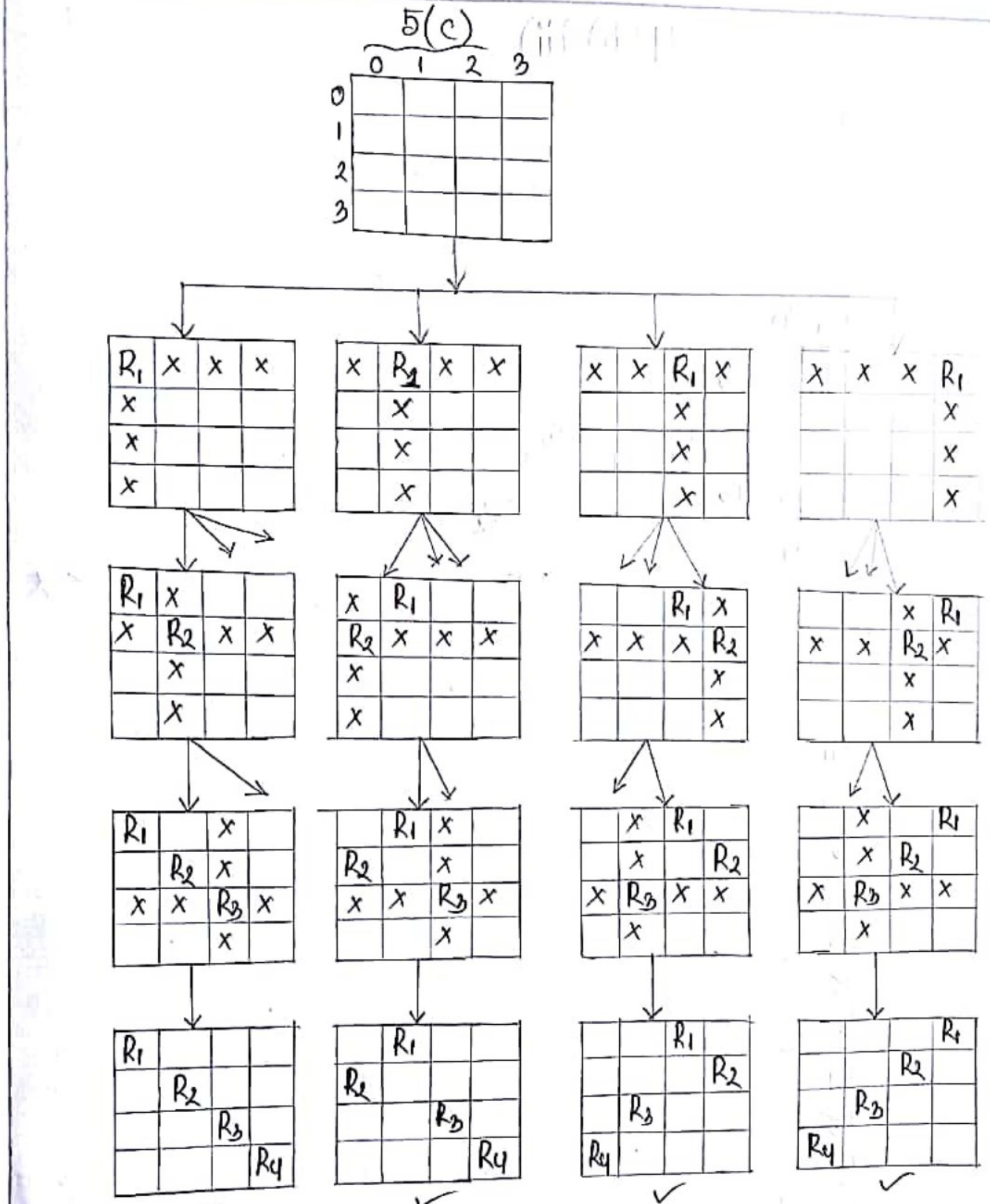


Fig: State Space tree of 4-Rook Problem.